

# AIMS AI Agent — Implementation Roadmap

---

## Phased Development Plan

Version: 1.0 (Architecture Draft)

Date: July 2026

---

## Overview

---

This roadmap breaks the AIMS AI Agent implementation into manageable phases. Each phase delivers working functionality that can be tested and validated before moving to the next.

---

## Phase 1: Foundation (Weeks 1-3)

---

**Goal:** Solid database, embeddings, and basic LLM integration

### Deliverables

#### 1. Database Setup

- PostgreSQL with pgvector extension
- Schema migration system (Prisma or Drizzle)
- Core tables: tenants, users, conversations, messages, knowledge\_sources, knowledge\_chunks

#### 2. Knowledge Base Migration

- Move static knowledge entries to database
- Create ingestion pipeline for PDFs
- Implement chunking strategy (512 tokens, 64 overlap)
- Generate embeddings using OpenAI text-embedding-3-small
- Store vectors in pgvector

#### 3. RAG Pipeline (Basic)

- Query → Embedding → Vector Search → Top-K chunks
- Simple prompt construction with chunks
- Basic citation extraction

#### 4. LLM Integration

- OpenAI GPT-4 client setup
- Streaming response support
- Token counting and limits
- Error handling and retries

#### 5. Environment Configuration

- Environment variables for all services
- Docker Compose for local development
- Basic health check endpoint

## Technical Tasks

```
[ ] Set up PostgreSQL + pgvector locally
[ ] Create Prisma schema with all core tables
[ ] Run initial migration
[ ] Create knowledge ingestion script
[ ] Migrate existing 9 entries to database
[ ] Generate embeddings for all chunks
[ ] Implement vector search query
[ ] Set up OpenAI client with streaming
[ ] Create basic chat API with RAG
[ ] Test end-to-end: query → retrieve → generate → respond
[ ] Add environment configuration
[ ] Create docker-compose.yml
```

## Success Criteria

- [ ] Can store knowledge chunks with embeddings in PostgreSQL
- [ ] Can search knowledge using semantic similarity
- [ ] Can generate responses using LLM with retrieved context
- [ ] Responses include proper citations
- [ ] Streaming works in the frontend
- [ ] Basic error handling in place

---

## Phase 2: Intelligence (Weeks 4-6)

---

**Goal:** Intent classification, confidence scoring, and enhanced retrieval

### Deliverables

#### 1. Intent Classification

- Rule-based classifier (start simple)
- Entity extraction (tax types, amounts, dates)
- Confidence scoring for classification

#### 2. Enhanced Retrieval

- Hybrid search (vector + BM25)
- Reciprocal Rank Fusion
- Category-based filtering
- Source authority weighting

#### 3. Confidence Scoring

- Multi-factor confidence calculation
- Human review triggers
- Review queue implementation

#### 4. Conversation Management

- Conversation history in database
- Context window management
- Session persistence

## 5. Enhanced Calculators

- Add CIT calculator
- Add Provisional Tax calculator
- Rule versioning system
- Calculation audit trail

## Technical Tasks

```
[ ] Implement intent classifier
[ ] Add entity extraction
[ ] Implement BM25 search
[ ] Add Reciprocal Rank Fusion
[ ] Implement confidence scoring
[ ] Create review queue table and API
[ ] Add conversation history API
[ ] Implement context window management
[ ] Add CIT calculator
[ ] Add Provisional Tax calculator
[ ] Add rule versioning
[ ] Test confidence scoring accuracy
[ ] Test review queue workflow
```

## Success Criteria

- [ ] System correctly classifies intent >80% of the time
- [ ] Hybrid search improves retrieval quality
- [ ] Confidence scores correlate with answer quality
- [ ] Low-confidence answers are flagged for review
- [ ] Conversation history persists across page reloads
- [ ] Calculators produce correct results with audit trail

---

## Phase 3: ERPNext Integration (Weeks 7-9)

---

**Goal:** Bidirectional integration with ERPNext/Frappe

### Deliverables

#### 1. Frappe Custom App

- App skeleton with hooks.py
- AIMS Chat DocType
- AIMS Audit Log DocType
- AIMS Settings DocType
- Whitelisted API methods

#### 2. Session Synchronization

- ERPNext session → AIMS authentication

- User mapping (Frappe user → AIMS user)
- Permission synchronization

### 3. Context-Aware Responses

- Detect current DocType and document
- Provide contextual help based on open module
- Link to relevant ERPNext pages

### 4. Data Access

- Read financial data from ERPNext
- Access chart of accounts
- Access transaction history
- Access employee data (with permissions)

### 5. Event Listeners

- Invoice submission triggers
- Journal entry triggers
- Compliance deadline sync

## Technical Tasks

```
[ ] Create Frappe app skeleton
[ ] Define AIMS Chat DocType
[ ] Define AIMS Audit Log DocType
[ ] Define AIMS Settings DocType
[ ] Implement whitelisted API methods
[ ] Implement session sync
[ ] Implement user mapping
[ ] Implement permission sync
[ ] Add ERPNext context detection
[ ] Implement data access layer
[ ] Add event listeners
[ ] Test ERPNext ↔ AIMS communication
[ ] Test context-aware responses
```

## Success Criteria

- [ ] Can authenticate via ERPNext session
- [ ] AIMS Chat UI works inside ERPNext
- [ ] AI provides contextual help based on open document
- [ ] Can read financial data from ERPNext
- [ ] Events from ERPNext trigger appropriate actions
- [ ] Audit logs are created in both systems

---

## Phase 4: Multi-Agent System (Weeks 10-12)

---

**Goal:** Specialized agents with coordinated responses

## Deliverables

### 1. Agent Framework

- Base agent interface
- Agent registry
- Tool system
- Coordination protocol

### 2. Specialized Agents

- Tax Advisory Agent (primary)
- Accounting Agent
- ERPNext Agent
- Compliance Agent

### 3. Tool System

- Calculator tools
- Knowledge search tools
- Document generation tools
- ERPNext API tools

### 4. Multi-Agent Coordination

- Sequential coordination
- Parallel coordination
- Response merging

## Technical Tasks

```
[ ] Design base agent interface
[ ] Implement agent registry
[ ] Implement tool system
[ ] Create Tax Advisory Agent
[ ] Create Accounting Agent
[ ] Create ERPNext Agent
[ ] Create Compliance Agent
[ ] Implement multi-agent coordination
[ ] Implement response merging
[ ] Test single-agent scenarios
[ ] Test multi-agent scenarios
[ ] Optimize agent response times
```

## Success Criteria

- [ ] Each agent specializes in its domain
- [ ] Tools are executed correctly by agents
- [ ] Multi-agent coordination works for complex queries
- [ ] Response merging produces coherent answers
- [ ] Agent response times meet targets

---

## Phase 5: Enterprise Features (Weeks 13-16)

---

**Goal:** Multi-tenant, security, compliance calendar, document drafting

## Deliverables

### 1. Multi-Tenant Architecture

- Tenant isolation
- Data partitioning
- Tenant-specific configurations

### 2. Security Enhancements

- JWT authentication (standalone mode)
- Role-based access control
- Sensitive data encryption
- Audit logging

### 3. Compliance Calendar

- Obligation tracking
- Deadline management
- Reminder system
- Filing evidence tracking

### 4. Document Drafting

- Template system
- TRA response letters
- Tax objection drafts
- Filing checklists

### 5. Admin Dashboard

- Knowledge source management
- User management
- Review queue management
- System health monitoring

## Technical Tasks

```
[ ] Implement tenant isolation
[ ] Add JWT authentication
[ ] Implement RBAC
[ ] Add encryption for sensitive fields
[ ] Create compliance calendar tables
[ ] Implement deadline tracking
[ ] Create reminder system
[ ] Build document template engine
[ ] Create TRA response template
[ ] Create tax objection template
[ ] Build admin dashboard
[ ] Add user management
[ ] Test multi-tenant isolation
[ ] Test RBAC enforcement
```

## Success Criteria

- [ ] Multiple tenants can use the system independently
- [ ] Users can only access permitted data
- [ ] Sensitive data is encrypted

- [ ] Compliance deadlines are tracked
- [ ] Documents can be generated from templates
- [ ] Admin can manage all aspects of the system

---

## Phase 6: Advanced AI (Weeks 17-20)

---

**Goal:** Document understanding, financial analysis, advanced features

### Deliverables

#### 1. Document Understanding

- Enhanced PDF parsing
- Table extraction
- OCR for scanned documents
- Document classification

#### 2. Financial Analysis

- Ratio analysis
- Trend analysis
- Risk assessment
- Executive summary generation

#### 3. Advanced Audit

- Anomaly detection
- Duplicate payment detection
- Control gap analysis
- Fraud indicator detection

#### 4. WhatsApp/SMS Integration

- Simplified chat interface
- Template-based responses
- Compliance reminders

#### 5. Performance Optimization

- Caching layer
- Query optimization
- Background job processing
- Load testing

### Technical Tasks

```
[ ] Enhance PDF parsing
[ ] Implement table extraction
[ ] Add OCR support
[ ] Implement document classification
[ ] Build ratio analyzer
[ ] Build trend analyzer
[ ] Implement risk assessment
[ ] Implement anomaly detection
[ ] Build duplicate payment detection
[ ] Set up WhatsApp integration
[ ] Implement caching layer
[ ] Optimize database queries
[ ] Add background job processing
[ ] Perform load testing
```

## Success Criteria

- [ ] Can extract tables from PDFs
- [ ] Can perform OCR on scanned documents
- [ ] Financial analysis provides actionable insights
- [ ] Anomaly detection identifies potential issues
- [ ] WhatsApp integration works for basic queries
- [ ] System handles expected load

---

## Phase 7: Production Ready (Weeks 21-24)

---

**Goal:** Testing, monitoring, documentation, deployment

### Deliverables

#### 1. Testing

- Unit tests (>80% coverage)
- Integration tests
- E2E tests
- Load tests

#### 2. Monitoring

- Application metrics
- LLM usage tracking
- Error tracking
- Performance monitoring

#### 3. Documentation

- API documentation
- User guide
- Admin guide
- Deployment guide



#### 4. Deployment

- Production Docker configuration
- CI/CD pipeline
- Backup strategy
- Disaster recovery plan

#### 5. Security Audit

- Penetration testing
- Vulnerability scanning
- Security documentation
- Compliance verification

### Technical Tasks

```
[ ] Write unit tests for core modules
[ ] Write integration tests for APIs
[ ] Write E2E tests for critical flows
[ ] Set up monitoring (OpenTelemetry)
[ ] Set up error tracking (Sentry)
[ ] Set up performance monitoring
[ ] Write API documentation
[ ] Write user guide
[ ] Write admin guide
[ ] Write deployment guide
[ ] Create production Docker config
[ ] Set up CI/CD pipeline
[ ] Implement backup strategy
[ ] Perform security audit
[ ] Load test the system
[ ] Deploy to production
```

### Success Criteria

- [ ] Test coverage >80%
- [ ] All critical paths tested
- [ ] Monitoring shows healthy system
- [ ] Documentation is complete
- [ ] Deployment is automated
- [ ] Security audit passed
- [ ] Load test passed

---

## Resource Requirements

### Development Team

| Role | Count | Responsibility |
|------|-------|----------------|
|------|-------|----------------|

|                       |                                  |
|-----------------------|----------------------------------|
| Lead Developer1       | Architecture, core development   |
| Full-Stack Developer2 | Frontend, API, integration       |
| AI/ML Engineer1       | RAG, embeddings, LLM integration |
| ERPNext Consultant1   | Frappe app, ERPNext integration  |
| QA Engineer1          | Testing, quality assurance       |

Infrastructure

|                       |                 |                 |
|-----------------------|-----------------|-----------------|
| Component             | Development     | Production      |
| PostgreSQL + pgvector | Local/Docker    | Managed service |
| Redis                 | Local/Docker    | Managed service |
| MinIO/S3              | Local/Docker    | Cloud storage   |
| OpenAI API            | Development key | Production key  |
| ERPNext               | Test site       | Production site |

Budget Estimate (Monthly)

|                        |                  |
|------------------------|------------------|
| Item                   | Cost Range       |
| OpenAI API             | \$500-2000       |
| Infrastructure         | \$500-1500       |
| Development (6 months) | \$50,000-100,000 |
| ERPNext customization  | \$10,000-20,000  |
| Testing & QA           | \$5,000-10,000   |

Risk Mitigation

|                        |        |                                                          |
|------------------------|--------|----------------------------------------------------------|
| Risk                   | Impact | Mitigation                                               |
| LLM hallucination      | High   | RAG grounding, citation verification, confidence scoring |
| ERPNext API changes    | Medium | Version pinning, abstraction layer, fallback             |
| Data security breach   | High   | Encryption, RBAC, audit logging, security testing        |
| Poor retrieval quality | High   | Hybrid search, reranking, human evaluation               |
| High API costs         | Medium | Caching, query optimization, model selection             |
| User adoption          | Medium | Training, documentation, gradual rollout                 |

Success Metrics

Technical Metrics

|                     |             |
|---------------------|-------------|
| Metric              | Target      |
| Response time (p95) | < 5 seconds |
| Retrieval accuracy  | > 85%       |
| Citation accuracy   | > 95%       |
| System uptime       | > 99.5%     |
| Test coverage       | > 80%       |

Business Metrics

|        |        |
|--------|--------|
| Metric | Target |
|--------|--------|

User satisfaction > 4.0/5.0  
Queries resolved without human > 70%  
Time saved per query > 50%  
Compliance deadline adherence > 95%  
User retention (monthly) > 80%

---

*This roadmap is a living document. Phases may be adjusted based on progress, feedback, and changing requirements.*